

Boosting

Date: March 17, 2026

Acknowledgements. These notes draw inspiration from Cornell University’s CS4780 lecture notes on Boosting.

Last class we looked at a procedure called Bagging that helps reduce variance of the model. In this class we will look at a procedure called boosting that helps reduce bias of the model. Boosting is perhaps the most successful paradigm to emerge out of learning theory that is very actively used even today. The core idea here is to *boost* simple learners that do a bit better than random guessing and combine them in a way to get a more complicated learner that performs very well. The question of whether this is possible was in fact posed by our very own Michael Kearns in a machine learning class project in 1988, and later answered in the affirmative by Rob Schapire in 1990.

Weak Learners. As a practical example of a class of weak learners, $\mathcal{H}$ might contain decision stumps, which are decision or regression trees consisting of a single split. In other words, a decision stump $h(x)$ that chooses to split on the first feature value $[x]_1$ being less than 5 might use the decision rule:

$$h(x) = \begin{cases} +1 & \text{if } [x]_1 < 5 \\ -1 & \text{if } [x]_1 \geq 5 \end{cases}$$

Decision stumps are learned by considering every possible split and picking the best one. The prediction of a boosted ensemble of trees $H_T(x)$ is given by a weighted sum of the prediction of a set of T decision stumps, $H_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$.

It may seem strange to consider boosting high bias decision trees when simply allowing a single tree more splits can always achieve zero training error assuming unique training input features. The problem with large depth single trees, however, is that it is remarkably easy to overfit with them: because the number of leaves in a decision tree increases exponentially with depth, it is quite easy to underfit with a tree of depth k and yet overfit with a tree of depth $k + 1$.

In contrast, boosted trees often overfit much more gradually. Each round adds only one shallow tree, so the complexity of the model grows in small steps rather than all at once. Moreover, each new tree is trained only to correct the residual mistakes of the current ensemble. As the ensemble improves, those remaining mistakes become more diffuse and harder for a single stump to capture, so later rounds usually make only modest corrections instead of dramatically changing the predictor.

We focus on limited depth trees as weak learners because they are the most widely used model for boosting. Indeed, one of the models we will look at in this lecture—gradient boosted regression trees (GBRT)—is one of the most widely deployed and successful models in the world, and is typically extremely good out of the box. With that said, our requirements of a weak learner to successfully boost are very limited. In the classification setting, we only need that it is possible for $h(x)$ to achieve greater than 50% training accuracy. In the regression setting, our requirements are

somewhat more technical, but essentially all we require is that given an existing model $H(x)$, there exists an α such that $H(x) + \alpha h(x)$ is more accurate than $H(x)$.

1 Functional Gradient Descent Perspective

At a high level, boosting is an iterative procedure. In iteration $t + 1$, we will add a new model $h_{t+1}(x)$ to the ensemble. In other words we replace $H_t(x) = \sum_{i=1}^T \alpha_i h_i(x)$ with $H_{t+1}(x) = H_t(x) + \alpha_{t+1} h_{t+1}(x)$.

This is why boosting can be viewed as *gradient descent in function space*. In ordinary gradient descent, we optimize a parameter vector and move in the direction of the negative gradient. Here, the object being optimized is the predictor H itself, and the update direction is another function h . Thus the weak learner h_{t+1} plays the role of a descent direction.

To pick $h_{t+1}(x)$, we minimize a loss function that measures the error of our current model:

$$\ell(H) = \frac{1}{n} \sum_{i=1}^n \ell(H(x_i), y_i)$$

In a true function space one might define the inner product by integrating over all inputs. Since in learning we only have the training set, we use the empirical inner product

$$\langle f, g \rangle = \sum_{i=1}^n f(x_i) g(x_i).$$

This lets us talk about whether a weak learner h points in a similar direction to the negative gradient of the loss.

In particular, in iteration t we seek to solve the following minimization problem:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \ell(H_t + \alpha_{t+1} h)$$

To solve this minimization problem iteratively, we take inspiration from gradient descent and form a Taylor approximation to the loss. This approximation is only accurate when the step size is small, so for now we think of α_{t+1} as a fixed small constant and focus on choosing the best direction h .

$$\ell(H_t + \alpha_{t+1} h) \approx \ell(H_t) + \langle \nabla_{H_t} \ell(H_t), \alpha_{t+1} h \rangle.$$

Here $\nabla_{H_t} \ell(H_t)$ is as follows:

$$\nabla_{H_t} \ell(H_t) = \begin{bmatrix} \frac{\partial \ell(H_t)}{\partial H_t(x_1)} \\ \vdots \\ \frac{\partial \ell(H_t)}{\partial H_t(x_n)} \end{bmatrix}$$

Substituting this in for $\nabla_{H_t} \ell(H_t)$ and noting that α_{t+1} is now treated as a constant with respect to the minimization:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \langle \nabla_{H_t} \ell(H_t), h \rangle = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n \frac{\partial \ell(H_t)}{\partial H_t(x_i)} h(x_i).$$

Let us define

$$r_i = \frac{\partial \ell(H_t)}{\partial H_t(x_i)}.$$

Then the search for the next weak learner becomes

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n r_i h(x_i).$$

1.1 Generic Boosting (AnyBoost)

This gives a general template for boosting. At iteration t , we compute the pseudo-responses $r_1, \dots, r_n$ from the current ensemble H_t , and then try to find a weak learner h whose predictions are as negatively correlated with r as possible. In other words, we seek a learning algorithm

$$\mathcal{A}(\{(x_1, r_1), \dots, (x_n, r_n)\}) = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n r_i h(x_i).$$

We do not need to solve this optimization perfectly. It is enough to find a weak learner h such that

$$\sum_{i=1}^n r_i h(x_i) < 0,$$

because then adding a small positive multiple of h decreases the loss to first order.

The rest of the lecture is about showing that for particular choices of loss function and weak learner class, this generic boosting template reduces to familiar learning problems. We will now see two important examples of this template: Gradient Boosted Regression Trees (GBRT) and AdaBoost.

2 Case Study I: Gradient Boosted Regression Trees (GBRT)

We now instantiate the generic boosting template with limited-depth regression trees as weak learners and a differentiable convex loss.

As a special case, let us consider the base class $\mathcal{H}$ be the class of decision stumps (or limited depth regression trees) and ℓ be any differentiable convex loss function.

The general boosting minimization problem is:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n r_i h(x_i)$$

where $r_i = \frac{\partial \ell(H_t)}{\partial H_t(x_i)}$.

It's not immediately obvious how to train a weak learner that minimizes the above sum. However, we can perform some algebraic manipulation to make things more convenient. First, define $u_i = -r_i$ and substitute it in to the above, and multiply the loss by a constant 2, which does not change the optimum:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n -2u_i h(x_i)$$

Second, we assume that $\sum_{i=1}^n h_{t+1}(x_i)^2$ is constant. This in general won't be true, but for regression trees we can enforce it by rescaling the output values of the leaves: if we multiply all predictions of h_{t+1} by a constant c , then we can divide α_{t+1} by the same constant and keep the product $\alpha_{t+1}h_{t+1}$ unchanged. So we are free to normalize h_{t+1} and focus only on its direction.

This lets us write the minimization as:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n -2u_i h(x_i) + h(x_i)^2.$$

Finally, noting that u_i does not depend on h_{t+1} in any way, u_i^2 is constant for all i . Thus, we can make one additional addition to the above loss:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n u_i^2 - 2u_i h(x_i) + h(x_i)^2.$$

This gives us:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n (u_i - h(x_i))^2.$$

This is a remarkable result: while we didn't know how to solve the original minimization problem for any loss, we do know how to solve the final minimization problem above! This final equation simply tells us to find the weak learner (e.g., limited depth decision tree) that minimizes the squared loss between the predictions $h(x_i)$ and u_i . This is already what the regression tree algorithm does. The only thing we have to change is to simply pass in the u_i values as labels rather than y_i .

In other words, to boost decision stumps, we just call our existing regression tree training algorithm on a dataset where we replace the original labels y_i with u_i . Once we have found a new weak learner h_{t+1} , in GBRT we choose α_t to be a small constant. Thus $\alpha_1 = \dots = \alpha_T$, and is chosen as a hyperparameter to the algorithm. This small step size is often called *shrinkage*: it regularizes the model by making each new tree only a conservative correction to the current ensemble, rather than allowing one tree to change the predictor too aggressively.

Squared Loss Example. As a concrete example, if the loss function is the squared loss $\ell(H) = \frac{1}{2} \sum_{i=1}^n (H(x_i) - y_i)^2$, then

$$u_i = -\frac{\partial \ell}{\partial H_t(x_i)} = y_i - H_t(x_i),$$

which is simply the *residual*—the difference between the true label and the current prediction. In this case, each iteration of GBRT fits a new regression tree to the residuals of the current ensemble. However, the result above holds for *any* differentiable convex loss: we always reduce to fitting a regression tree on the pseudo-labels u_i .

Algorithm 1: Gradient Boosted Regression Trees (GBRT)

Input: Training data $\{(x_1, y_1), \dots, (x_n, y_n)\}$, iterations T , step size α , differentiable loss ℓ
Initialize $H_0(x) = 0$;
for $t = 0, 1, \dots, T - 1$ **do**
 Compute pseudo-labels: $u_i = -\frac{\partial \ell}{\partial H_t(x_i)}$ for all i ;
 Train regression tree: $h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n (h(x_i) - u_i)^2$;
 Update ensemble: $H_{t+1}(x) = H_t(x) + \alpha \cdot h_{t+1}(x)$;
end
return $H_T(x)$

3 Case Study II: AdaBoost

We now instantiate the same boosting template for binary classification, using binary weak learners and the exponential loss:

$$\ell_{\text{exp}}(H) = \sum_{i=1}^n \exp(-y_i H(x_i))$$

In this case, we have:

$$\frac{\partial \ell_{\text{exp}}(H_t)}{\partial H_t(x_i)} = -y_i \exp(-y_i H_t(x_i))$$

To find the next weak learner, we thus need to solve the optimization problem below, which is a special case from the general boosting minimization problem:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} - \sum_{i=1}^n y_i \exp(-y_i H_t(x_i)) h(x_i)$$

Define $w_i = \frac{1}{Z} \exp(-y_i H_t(x_i))$, where $Z = \sum_{i=1}^n \exp(-y_i H_t(x_i))$ so that $\sum_{i=1}^n w_i = 1$. Note that Z is entirely independent of $h_{t+1}(x_i)$. Thus, the above minimization problem becomes:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} - \sum_{i=1}^n w_i y_i h(x_i).$$

Next, note that $y_i h(x_i)$ is positive when h classifies x_i correctly, and is negative when it classifies it incorrectly:

$$y_i h(x_i) = \begin{cases} -1 & h(x_i) \neq y_i \\ 1 & h(x_i) = y_i. \end{cases}$$

Thus, we can replace the above minimization problem with a minimization problem over two distinct sums:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i: h(x_i) \neq y_i} w_i - \sum_{i: h(x_i) = y_i} w_i.$$

Since the second term is one minus the first term, up to constants this minimization problem is:

$$h_{t+1} = \arg \min_{h \in \mathcal{H}} \sum_{i: h(x_i) \neq y_i} w_i = \arg \min_{h \in \mathcal{H}} \text{err}_t.$$

This is the *weighted classification error* err_t . Thus, $h_{t+1}(x)$ can be taken to be a weak learner that reduces the weighted classification error. In other words, if we augment our decision tree algorithm to take in weighted training examples using weights w_i , then boosting the exponential loss becomes equivalent to using the algorithm on the weighted points to add a new tree.

Remark. As long as $\mathcal{H}$ is *negation closed* (i.e., for every $h \in \mathcal{H}$ we also have $-h \in \mathcal{H}$), the weighted error cannot exceed $\frac{1}{2}$. The reason is that if h has error err_t , then $-h$ has error $1 - \text{err}_t$. Since h was chosen to minimize the error, we must have $\text{err}_t \leq 1 - \text{err}_t$, i.e., $\text{err}_t \leq \frac{1}{2}$. CART trees are negation closed, so this is generally satisfied. When $\text{err}_t = \frac{1}{2}$, the weak learner is no better than a coin toss and boosting terminates.

3.1 Optimal Step Size α

Unlike GBRT where α is a fixed hyperparameter, in AdaBoost the optimal step size has a closed-form expression:

$$\alpha = \frac{1}{2} \ln \frac{1 - \text{err}_t}{\text{err}_t}.$$

Note that $\alpha > 0$ when $\text{err}_t < \frac{1}{2}$, meaning we place positive weight on classifiers better than random guessing. As $\text{err}_t \rightarrow 0$, $\alpha \rightarrow \infty$: we trust very accurate classifiers heavily. As $\text{err}_t \rightarrow \frac{1}{2}$, $\alpha \rightarrow 0$: a coin-toss classifier contributes nothing.

3.2 Weight Update Rule

After fitting a weak learner h_t and choosing its weight α_t , AdaBoost updates the training distribution by

$$w_{t+1,i} = \frac{w_{t,i}}{Z_t} \exp(-\alpha_t y_i h_t(x_i)),$$

where

$$Z_t = \sum_{j=1}^n w_{t,j} \exp(-\alpha_t y_j h_t(x_j))$$

is the normalizing constant that ensures the new weights sum to 1. If x_i is classified correctly, then $y_i h_t(x_i) = 1$ and its weight is scaled down; if it is misclassified, then $y_i h_t(x_i) = -1$ and its weight is scaled up. Thus AdaBoost increases the weight on hard examples and decreases the weight on easy ones.

3.3 AdaBoost Algorithm

Algorithm 2: AdaBoost

Input: Training data $\{(x_1, y_1), \dots, (x_n, y_n)\}$ with $y_i \in \{-1, +1\}$, iterations T

Initialize weights: $w_i = \frac{1}{n}$ for all i ;

for $t = 1, \dots, T$ **do**

 Train weak learner on weighted data: $h_t = \arg \min_{h \in \mathcal{H}} \sum_{i: h(x_i) \neq y_i} w_i$;

 Compute weighted error: $\text{err}_t = \sum_{i: h_t(x_i) \neq y_i} w_i$;

if $\text{err}_t \geq \frac{1}{2}$ **then stop**;

 Compute step size: $\alpha_t = \frac{1}{2} \ln \frac{1 - \text{err}_t}{\text{err}_t}$;

 Compute normalizer: $Z_t = \sum_{i=1}^n w_i \exp(-\alpha_t h_t(x_i) y_i)$;

 Update weights: $w_i \leftarrow \frac{w_i}{Z_t} \exp(-\alpha_t h_t(x_i) y_i)$ for all i ;

end

return $H_T(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

3.4 Convergence Guarantee

If the weak learner does at least slightly better than random guessing in every round, then AdaBoost's training error decreases exponentially fast.

Theorem 3.4.1. *Let H_T be the output of AdaBoost after T steps, and suppose that $\frac{1}{2} - \text{err}_t \geq \gamma > 0$ for all t . Then*

$$\hat{R}(H_T) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{sign}(H_T(x_i)) \neq y_i] \leq \exp(-2\gamma^2 T).$$

We will prove this theorem in the next homework.

3.5 Generalization Performance of AdaBoost

Note that the training error goes down exponentially fast. It also seems that the classifier we get with more iterations gets more complex. This would imply that even though we reduce bias, we are increasing variance, leading to overfitting. However, more often than not, we do not observe any overfitting as T gets larger. See Figure 1 (left) for an example of a setup where no overfitting is observed. In fact, test error continues to drop even after training error has stopped improving.

Why does this happen? Well, the answer is *margins*. Much like we saw in SVM, here the boosting algorithm implicitly encourages a large margin under a slightly different notion of margin. For a training point (x_i, y_i) , the normalized margin is

$$\frac{y_i \sum_{t=1}^T \alpha_t h_t(x_i)}{\sum_{t=1}^T \alpha_t},$$

which is exactly the total weight of correct classifiers minus the total weight of incorrect classifiers, normalized by the total weight. If this quantity is large and positive, we are very confident in the prediction; if it is negative, the prediction is incorrect. Observe that in Figure 1 (right), the margin

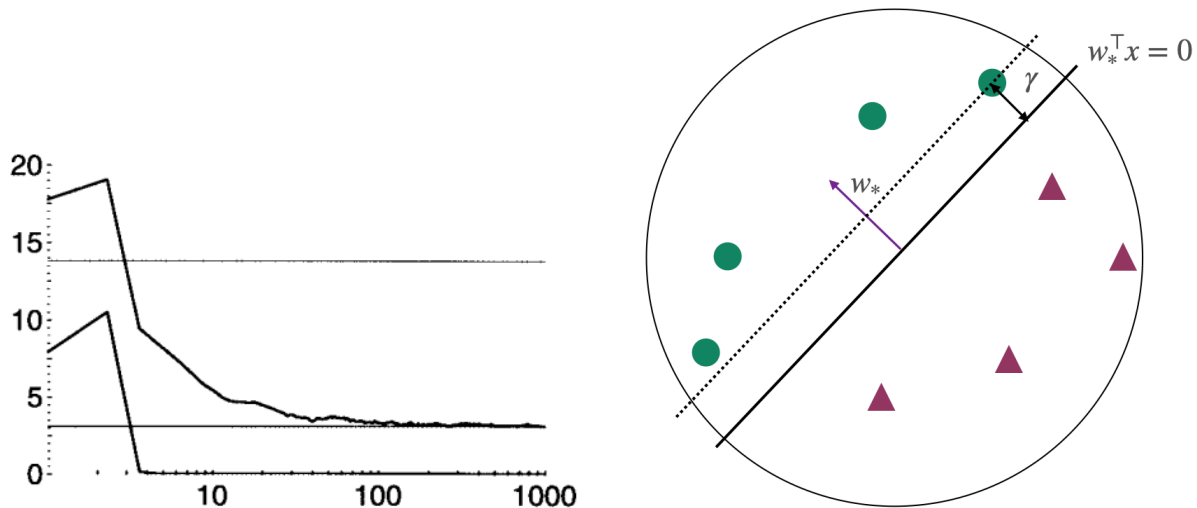


Figure 1: From Schapire et al'98 showing that boosting does not overfit, due to maximizing margin. (*left*) x-axis is number of rounds, y-axis is error (in %). The top curve is test error and the lower curve is training error. (*right*) x-axis is margin, y-axis is cumulative distribution. The three curves correspond to different training iterations.

for all points is at least 0.5. This helps explain why AdaBoost can continue to generalize well even after the training error has already dropped to zero.

Note: This phenomenon is also observed in neural networks where overfitting does not imply poor generalization.